Homework 3: Robot Manipulation and Semantic Robot Knowledge

Human-centered Intelligent System Lab
National Yang Ming Chiao Tung University

Fall 2026

1 Introduction

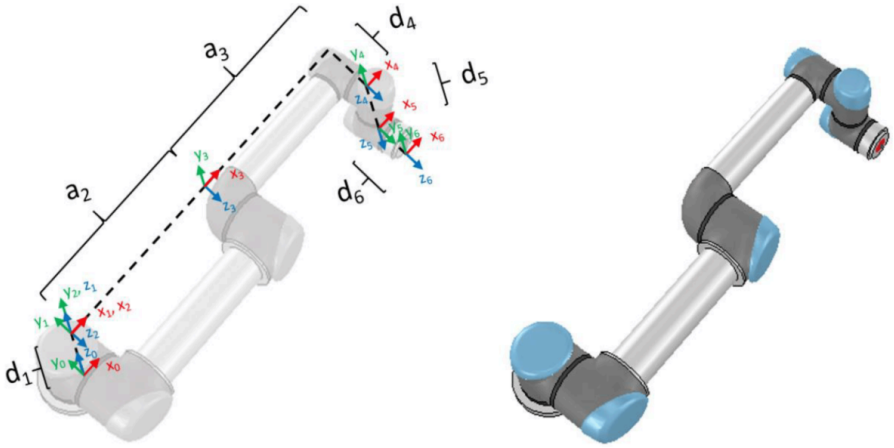


Figure 1: The 6-degree-of-freedom robot arm.

Robot manipulation requires a robot to reason about the relationship between its joint configuration and the pose of its end-effector. Forward Kinematics (FK) computes the end-effector pose from a given joint configuration, while differential kinematics characterizes how joint motion affects end-effector motion through the Jacobian. Building on these foundations, Inverse Kinematics (IK) determines joint configurations that achieve a desired end-effector pose, often requiring numerical methods to solve the underlying nonlinear problem [1]. Together, these techniques establish the mathematical basis for motion planning and manipulation tasks such as reaching, grasping, and transporting objects. Physics-based simulation environments, such as PyBullet, further support the development and evaluation of robotic manipulation systems [2].

In this homework, students progress from kinematic analysis to integrated robot manipulation. They first implement and analyze FK and its associated Jacobian, then develop an iterative IK solver to reach target poses. These components are subsequently integrated into a State Machine (SM) to execute a pick-and-place task in simulation. Through this process, students evaluate manipulation outcomes, investigate potential failure sources, and distinguish numerical errors in kinematic computation from task-level execution failures involving motion, contact, grasping, and placement.

Reliable robotic systems also require structured representations of robot specifications, kinematic configurations, and execution information. This homework therefore introduces ontology-based modeling and the Resource Description Framework (RDF), which represents structured information as a graph of subject–predicate–object statements [3]. The Core Ontology for Robotics and Automation (CORA)

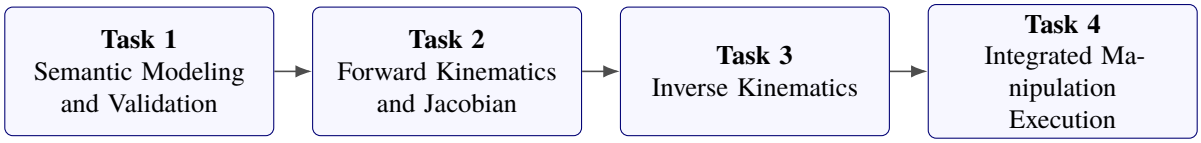
provides a robotics-oriented conceptual foundation for representing entities and their relationships [4]. In addition, Quantities, Units, Dimensions and Types (QUDT) provides a vocabulary for describing quantities and units in a structured and interoperable manner [5]. These representations help organize robot specifications and kinematic data so that they can be interpreted and checked consistently.

To validate the represented knowledge, students use the Shapes Constraint Language (SHACL), a W3C Recommendation for expressing constraints over RDF graphs and validating graph data against those constraints [6]. Students formulate and check structural and cross-entity consistency requirements, then use validation results as diagnostic evidence when investigating potential problems in the manipulation pipeline. Semantic validation supports the interpretation and diagnosis of system information, but does not replace numerical kinematic computation or independently guarantee physical correctness.

By combining mathematical modeling, numerical problem-solving, state-based task execution, and semantic validation, this homework provides an integrated workflow for developing and evaluating robotic manipulation systems. Students should be able to explain how kinematic computations influence downstream execution, how semantic constraints reveal inconsistencies in represented data, and how evidence from these different layers can be used together to diagnose failures.

2 Structure and Workflow

This assignment develops a robotic manipulation pipeline through four stages: semantic modeling and validation, forward kinematics, inverse kinematics, and integrated task execution. Each stage addresses a different aspect of the system, from representing robotic knowledge to computing motion and evaluating task-level behavior.



2.1 Workflow Overview

1. **Semantic modeling and validation.** Represent robot specifications and kinematic execution records as structured, machine-readable knowledge. Validate the resulting representation to identify missing structural information and violations of numerical constraints.
2. **Forward kinematics and differential kinematics.** Establish the relationship between joint configurations and end-effector motion. Compute the end-effector pose and geometric Jacobian to describe both the robot's configuration and its local motion behavior.
3. **Inverse kinematics.** Given a target end-effector pose, iteratively determine a joint configuration that reduces the position and orientation errors while respecting the robot's joint limits. The FK and Jacobian developed in the previous stage provide the basis for this numerical solution.
4. **Integrated manipulation execution.** Execute the provided pick-and-place state machine using the implemented kinematic components. Evaluate the resulting behavior through motion tracking, consistency checks, contact, and final placement, and use execution diagnostics to identify potential failure sources.

Together, these stages connect structured robot knowledge, mathematical motion computation, and task-level execution. Individual component checks help establish correctness, while the integrated manipulation pipeline evaluates whether the components work together as intended.

2.2 Environment Setup

The assignment uses Pixi to manage the required software environment. Pixi is a cross-platform package and environment manager; refer to its official installation documentation for installation instructions [7].

The provided `pixi.toml` and `pixi.lock` specify the dependencies needed for semantic validation and robotic simulation. A GPU is not required.

Install Pixi if it is not already available, then initialize the environment from the assignment directory:

```
curl -fsSL https://pixi.sh/install.sh | bash
cd hw3_template
pixi install
```

Run the assignment commands within the Pixi environment. The commands and expected results for each task are described in the corresponding sections below.

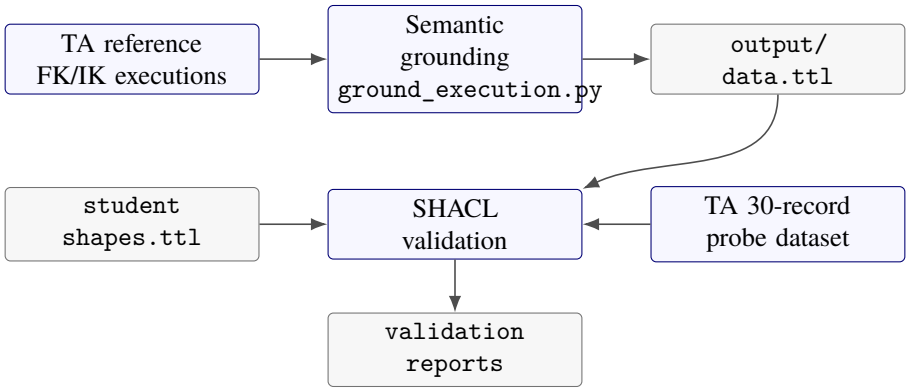
The first execution of Task 1 downloads Apache Jena and caches it under `semantic/.cache/`. Network access is required for the initial environment setup and Jena download; subsequent executions can use the cached files.

The assignment is officially supported on Linux (Ubuntu 20.04 or newer). Windows and macOS are not officially supported.

2.3 Task 1: Semantic Grounding and SHACL Validation (30 points)

2.3.1 Execution Model and Workflow

Convert numerical FK/IK execution records into a semantic graph, then use SHACL to detect problem states directly from the stored numbers:



The design follows the REUSE principle: a robot is a `cora:Robot`; joints and joint states use SOMA; poses use `soma:6DPose` and IEEE 1872 POS; units use QUDT IRIs. The local `hw3:` vocabulary is only for concepts not covered by those standards. Numeric values must be structured nodes and typed `xsd:double` literals, never JSON strings.

2.3.2 Mathematical Representation

Let an RDF data graph be a finite set of triples

$$G \subseteq \mathcal{N} \times \mathcal{P} \times (\mathcal{N} \cup \mathcal{L}),$$

where $\mathcal{N}$, $\mathcal{P}$, and $\mathcal{L}$ are the sets of nodes, properties, and literals. We use the predicate notation

$$C(x) \iff (x, \mathsf{rdf:type}, C) \in G, \qquad p(x,y) \iff (x,p,y) \in G.$$

Grounding is a serialization function

$$\gamma : \mathcal{E} \to 2^G, \qquad G_{\text{exec}} = G_{\text{ontology}} \cup \bigcup_{e \in \mathcal{E}} \gamma(e),$$

which maps each robot specification or execution record e to RDF triples.

It must preserve structure: a six-joint configuration c satisfies

$$\mathsf{JointConfiguration}(c) \quad \wedge \quad |\{s : \mathsf{hasJointState}(c, s)\}| = 6,$$

and every linked state carries its joint identity and numerical angle:

$$\forall s\,[\mathsf{hasJointState}(c, s) \Rightarrow \exists j, v\ (\mathsf{isStateOfJoint}(s, j) \wedge \mathsf{hasJointPosition}(s, v))].$$

A grounded IK execution x similarly connects one robot, target, output configuration, status, residual r_x, and target distance d_x.

For shapes graph S and data graph G, define validation as

$$\mathrm{Report}(S, G) = \{(x, m, p, v) : x \text{ violates a constraint with message } m \text{ on path } p \text{ at value } v\}.$$

The graph conforms exactly when

$$\mathrm{conforms}(S, G) \iff \mathrm{Report}(S, G) = \emptyset.$$

The four numerical course constraints are

$$\begin{aligned}
\Phi_{\mathrm{FK}} &: \quad e_x \le 0.005, && x : \mathsf{FKComputation},\\
\Phi_{\mathrm{AR}} &: \quad d_x \le 0.90, && x : \mathsf{IKComputation},\\
\Phi_{\mathrm{NC}} &: \quad r_x \le 0.02, && x : \mathsf{IKComputation},\\
\Phi_{\mathrm{JL}} &: \quad \ell_j \le q_s \le u_j, && s : \mathsf{JointState},\ \mathsf{isStateOfJoint}(s, j).
\end{aligned}$$

Violation produces, respectively, `FK_INACCURATE`, `ARM_OUT_OF_RANGE`, `NO_CONVERGENCE`, or `JOINT_LIMIT_VIOL`. All inequalities are inclusive: equality with a threshold or joint bound conforms.

2.3.3 Grounding Requirements

Read the worked example `fk_computation_to_triples()` and the serializers in `semantic/common.py`. Implement:

1. `robot_spec_to_triples()`: create `stu:my_ur5` as a `cora:Robot`, record its producer and six DoF, and link six `soma:RevoluteJoint` instances. Each joint must contain its index, D–H parameters a, d, α, and inclusive lower/upper limits.
2. `ik_computation_to_triples()`: create one `hw3:IKComputation` with provenance, robot, target, status, residual, target distance, and a structured six-state joint configuration. Use `common.joint_config_to_tt`

2.3.4 SHACL Requirements

The provided FK accuracy shape is a worked example: pose error must be at most 0.005 m. Add the following three shapes:

Condition	Legal range	Required message prefix
Target distance	$d \le 0.90$ m	`ARM_OUT_OF_RANGE:`
IK residual	$r \le 0.02$ m	`NO_CONVERGENCE:`
Joint position	$q_{\min} \le q \le q_{\max}$	`JOINT_LIMIT_VIOLATION:`

Use `sh:maxInclusive` for the first two constraints. The third must be a SHACL–SPARQL constraint on `soma:JointState`, because its position and the linked joint's bounds occur on different nodes. Its filter must use strict `<` and `>` comparisons; equality with a bound is legal.

2.3.5 Run and Expected Results

```
bash semantic/run_task1.sh --group <your-group-id>

# after Tasks 2 and 3:
bash semantic/run_task1.sh --group <your-group-id> --own
```

The command checks toolchains, prepares Jena, grounds the data, runs three validations, and scores the result. A correct implementation has no grounding structure violations, flags `target_mid` and `target_far` as out of range, leaves `target_near` clean, reports no joint-limit violation in the reference run, and matches all 30 records in `ta-answer-key.json`. The expected Task 1 score is 30/30.

2.4 Task 2: Forward Kinematics (20 points)

Implement `your_fk(DH_params, q, base_pos)` in `fk.py`. Its inputs are the six classic D–H parameter records, six joint angles, and a base position. It returns the seven-dimensional end-effector pose

$$[x, y, z, q_x, q_y, q_z, q_w]$$

and the 6×6 geometric Jacobian.

For each revolute joint, use the classic D–H transform

$$^{i-1}T_i = R_z(\theta_i)T_z(d_i)T_x(a_i)R_x(\alpha_i).$$

Save the preceding-frame origin p_{i-1} and z axis z_{i-1}. For end-effector position p_e, Jacobian column i is

$$J_i = \begin{bmatrix} z_{i-1} \times (p_e - p_{i-1}) \\ z_{i-1} \end{bmatrix}.$$

The D–H table in `get_ur5_DH_params()` is authoritative and intentionally differs slightly from the official UR5 specification. Do not use PyBullet or another library to directly compute FK/Jacobians, and do not modify the final `adjustment` block.

```
python fk.py              # visible easy / medium / hard tests
python fk.py -g -vp       # optional GUI and pose visualization
```

Success means zero FK and Jacobian error counts for every visible test file. The post-score semantic diagnosis should report conforming sampled records.

2.5 Task 3: Inverse Kinematics (40 points)

Implement `your_ik()` in `ik.py`. Warm-start from the current joint configuration, repeatedly call your own FK/Jacobian, and reduce a six-vector containing position and orientation error. A robust recommended update is damped least squares:

$$\Delta q = J^T \left(JJ^T + \lambda^2 I\right)^{-1} \Delta x, \qquad q \leftarrow \text{clip}(q + s\Delta q, q_{\min}, q_{\max}),$$

where λ is damping and s is the step rate.

A rotation vector of $R_{\text{target}}R_{\text{current}}^T$ provides a useful orientation error. Stop when the error norm is below the threshold or the iteration budget is exhausted.

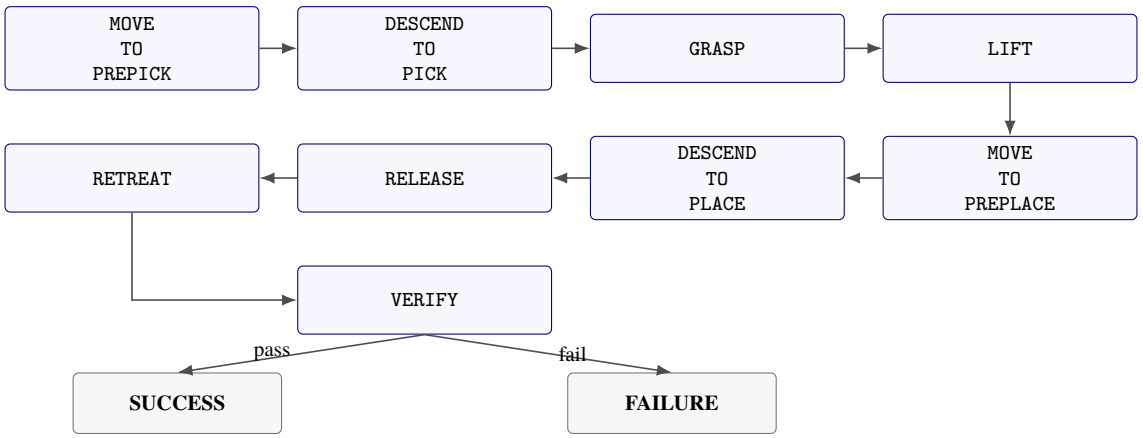
The implementation must not call PyBullet IK or another library's IK solver. `pybullet_ik()` is for behavioral comparison only. Grading invokes your function with default arguments, so tune the defaults rather than relying on command-line changes.

```
python ik.py
```

Aim for zero error counts on the easy, medium, and hard cases, mean positional error near 10^{-3} m, and conforming semantic diagnosis records.

2.6 Task 4: FSM Manipulation Pipeline (10 points)

No code change is required. The provided FSM runs ten seeded pick-and-place episodes through



Every motion state solves the target using your IK and requires the achieved end-effector position to be within 3 cm. It also evaluates your FK on measured joint angles and requires agreement with the simulator within 1 cm. The pick descent continues until suction contact; verification requires the block to rest within 6 cm of the goal.

```
python fsm_task.py
python fsm_task.py -g       # optional PyBullet visualization
```

The expected result is ten `SUCCESS` episodes and 10/10 points. Failures name both the FSM state and reason, such as `IK_MISS`, `FK_MISMATCH`, `NO_CONTACT`, or `PLACE_MISS`; the semantic layer additionally reports applicable problem flags.

3 Grading

The assignment is worth 100 points in total. Grading covers semantic modeling and validation, forward kinematics, inverse kinematics, and integrated manipulation execution. Each task is evaluated according to its required functionality and the corresponding verification results.

Task	**Points**
Task 1: Semantic Grounding and SHACL Validation	30
Task 2: Forward Kinematics	20
Task 3: Inverse Kinematics	40
Task 4: FSM Manipulation Pipeline	10
Total	**100**

The implementation and its supporting evidence must satisfy the requirements specified in each task. Automated evaluation scripts may be used to verify structural correctness, numerical accuracy, constraint detection, and execution behavior. The report and `README.md` must document the implementation and provide sufficient information to reproduce the results.

3.1 Assessment Rubric

3.1.1 Task 1: Semantic Grounding and SHACL Validation (30 points)

Task 1 evaluates the conversion of robot specifications and execution records into structured RDF representations and the use of SHACL to validate structural and numerical constraints.

- **S1 – Grounding Structure (12 points):** The robot specification and IK execution records must be represented using the required semantic vocabulary and relationships. The robot must be linked to six revolute joints, and each joint state must be represented as a structured resource with its joint identity and numerical position. Required types, properties, provenance, and numeric datatypes must be present.
- **S2 – Constraint Detection (8 points):** The required SHACL shapes must detect out-of-range targets, non-convergent IK results, and joint-limit violations. The provided FK accuracy constraint must remain functional. The validation results must agree with the specified threshold behavior.
- **S3 – Probe Dataset Agreement (10 points):** The implementation must produce the expected validation outcomes for the provided 30-record probe dataset, including clean records, faulty records, and boundary cases. The required violation messages and classifications must match the specification.

A Turtle graph that parses successfully does not necessarily satisfy the grounding requirements. Structural completeness, relationship correctness, datatype correctness, and constraint behavior are all part of the assessment.

3.1.2 Task 2: Forward Kinematics (20 points)

Task 2 evaluates the student's implementation of FK and the geometric Jacobian in `fk.py`.

- **Forward Kinematics (10 points):** The implementation must compute the end-effector position and orientation from the six joint angles using the provided classic D–H parameters and the required transformation convention.
- **Geometric Jacobian (10 points):** The implementation must compute the 6×6 geometric Jacobian using the preceding-frame origins and joint axes. Both its linear and angular components must follow the specified row ordering.

The implementation must not use PyBullet or another external library to directly calculate the required FK or Jacobian. The provided `adjustment` block must remain unchanged. Correctness is assessed using the prescribed test cases and numerical error criteria.

3.1.3 Task 3: Inverse Kinematics (40 points)

Task 3 evaluates the iterative numerical IK solver implemented in `ik.py`.

- **Numerical Solver Correctness:** The implementation must use the student's FK and Jacobian to iteratively reduce both position and orientation errors. It must not call PyBullet IK or another library's IK solver.
- **Joint Limits and Initialization:** The solver must warm-start from the supplied joint configuration and respect the provided joint limits throughout the iterative update process.
- **Convergence and Accuracy:** The solver should reach the specified target poses within the required error tolerance and iteration budget. Evaluation considers the prescribed easy, medium, and hard test cases.
- **Robustness:** The implementation must behave consistently across the prescribed cases and return meaningful results when the target cannot be reached within the allowed tolerance or iteration budget.

The official evaluation script and released test criteria determine the numerical score. The reference values in the task description are intended to guide development; do not replace the official evaluation criteria with an independently invented threshold.

3.1.4 Task 4: FSM Manipulation Pipeline (10 points)

Task 4 evaluates the integration of the student's FK and IK implementations into the provided finite-state-machine manipulation pipeline.

- **Execution Success:** The pipeline must complete the prescribed seeded pick-and-place episodes and satisfy the provided motion-tracking, FK consistency, contact, and placement checks.
- **Integration Correctness:** The implementation must work with the provided FSM without bypassing its checks or modifying the protected task logic.
- **Failure Diagnosis:** When an episode fails, the reported state and reason should be used to distinguish kinematic tracking errors from FK mismatches, contact failures, and placement failures.

The target result is ten successful episodes. The official evaluation script determines the awarded points. Students must not modify `fsm_task.py` to bypass the required checks.

3.2 Report Format

Submit a concise report in English as `report.pdf`. The report must explain the implementation, present the required verification evidence, and discuss the behavior of the complete manipulation pipeline.

The report should contain the following sections.

- **Implementation**
  Explain the main implementation decisions for semantic grounding, SHACL validation, FK, IK, and the integrated manipulation pipeline. Describe the numerical methods and data representations used, and identify any implementation difficulties and their solutions.
- **Verification and Results**
  Present the required task outputs and validation evidence. For Task 1, explain the RDF structure and summarize the SHACL validation results, including the required probe dataset. For Task 2 and Task 3, report the numerical results for the prescribed test cases. For Task 4, report the number of successful episodes and summarize any failure states or diagnostic messages.
- **Discussion**
  Discuss how errors in FK or IK can propagate to the manipulation pipeline. Explain the roles and limitations of semantic validation, numerical tests, and task-level execution checks. Where relevant, distinguish structural inconsistencies from numerical inaccuracies and physical execution failures.

The report must be consistent with the submitted source code and observed execution results. Do not claim that a task passed unless the corresponding output or verification evidence supports that claim.

3.3 Submission Package

Submit `<student-id>_hw3.zip` to the designated submission platform. Place the required files at the archive root or at the specified relative paths. Do not add an unnecessary enclosing directory.

The archive should contain the following materials:

```
<student-id>_hw3.zip
|-- fk.py
|-- ik.py
|-- fsm_task.py
|-- semantic/
|   |-- ground_execution.py
|   |-- common.py
|   |-- shapes.ttl
|   `-- other required semantic source files
|-- pixi.toml
|-- pixi.lock
|-- report.pdf
`-- README.md
```

The structure above illustrates the expected organization. Preserve any additional source files required by the released template, and follow the actual template paths if they differ from this illustration. Do not create placeholder files for modules that do not exist in the provided template.

The submission must satisfy the following requirements:

- **Implementation Files:** Include all modified source files needed to execute Tasks 1–4, including the required FK, IK, and semantic grounding or validation implementations.
- **Semantic Constraints:** Include the student-authored SHACL shapes and any other required semantic files that are part of the assignment template.
- **Environment Configuration:** Include the provided `pixi.toml` and `pixi.lock` files if they are part of the released template. Do not replace the prescribed environment with an undocumented local configuration.
- **Report:** Include one English PDF report describing the implementation, verification results, and discussion.
- **README:** Include execution instructions, the required environment setup, the commands used to run the tasks, and any relevant assumptions. The README must not depend on undocumented local files or paths.

Do not include hidden evaluation data, reference answers, cached third-party downloads, local virtual environments, generated output directories, or unrelated build artifacts unless the assignment explicitly requires them.

The submitted source files must be sufficient to reproduce the required workflow using the prescribed environment and the provided assignment inputs. Do not modify protected grading scripts or reference data to make the implementation pass.

3.4 Submission Rules

Submission deadlines and specific penalty policies are determined by the course and the student's institution. Follow the announcements and instructions provided by the designated submission platform.

- **Deadline:** Submit the assignment by the announced deadline.
- **Late Submission:** Late submissions are subject to the policy announced by the course or institution.
- **Submission Format:** Follow the required archive name, directory structure, source file paths, and report format. Incorrect or incomplete packaging may prevent the submission from being evaluated.
- **Reproducibility:** The submitted implementation must run in the prescribed environment. Document any required setup steps and avoid undeclared dependencies or machine-specific paths.
- **Academic Integrity:** Submitted work must comply with the course's academic-integrity policy. Unauthorized copying, sharing of reference solutions, or attempts to bypass the grading checks will be handled according to institutional policy.

4 FAQ

What is the total score?
The assignment is worth 100 points: Task 1 is worth 30 points, Task 2 is worth 20 points, Task 3 is worth 40 points, and Task 4 is worth 10 points.

Which environment should I use?
Use the provided Pixi environment and the released `pixi.toml` and `pixi.lock` files. The assignment is officially supported on Linux, Ubuntu 20.04 or newer.

What should I submit?
Submit `<student-id>_hw3.zip` containing the required implementation files, semantic constraint files, environment configuration, `report.pdf`, and `README.md`. Follow the actual directory structure of the released template.

Should I include generated validation reports?
Include them in the report when they are required as evidence, but do not add generated output directories or cached artifacts to the ZIP unless the assignment explicitly requests them.

Can I use PyBullet to calculate FK or the Jacobian?
No. Task 2 requires your own FK and Jacobian implementation. Use the prescribed D–H convention and retain the provided adjustment block.

Can I use a library's IK solver?
No. Task 3 requires your own iterative IK solver based on your FK and Jacobian. The provided PyBullet IK function is for behavioral comparison only.

Must I use damped least squares?
Damped least squares is the recommended method. Follow the required solver interface, joint limits, position/orientation error definitions, and official evaluation criteria. Do not replace the required implementation with an external IK solver.

What if SHACL reports a violation?
Inspect the affected resource, property path, numerical value, threshold, and violation message. Determine whether the problem is missing or malformed RDF data, a genuine numerical violation, or an incorrectly defined shape.

What if FK and IK tests pass but the FSM fails?
Inspect the first failing FSM state. Distinguish target tracking, FK consistency, contact, and final placement failures before changing parameters.

Can I modify the FSM to make all episodes succeed?
No. The provided FSM and its checks must remain intact. The objective is to integrate and evaluate your FK and IK implementations, not to bypass the task requirements.

What should the report include?
Explain the semantic model and SHACL results, FK/Jacobian verification, IK method and test results, and FSM episode outcomes. Discuss how numerical and structural errors can propagate to task-level execution.

What should I do if the provided command fails?
Record the command and complete error message, confirm that the Pixi environment is active, and check the required dependencies and input paths. Include relevant reproducibility information in your README or report.

References

[1] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2005, ISBN: 9780201543612.

[2] E. Coumans and Y. Bai, *Pybullet, a python module for physics simulation for games, robotics and machine learning*, GitHub repository, 2021. Accessed: Aug. 31, 2026. [Online]. Available: https://github.com/bulletphysics/bullet3

[3] R. Cyganiak, D. Wood, and M. Lanthaler, *Rdf 1.1 concepts and abstract syntax*, W3C Recommendation, 2014. [Online]. Available: https://www.w3.org/TR/rdf11-concepts/

[4] E. Prestes et al., "Towards a core ontology for robotics and automation," *Robotics and Autonomous Systems*, vol. 61, no. 11, pp. 1193–1204, 2013. DOI: 10.1016/j.robot.2013.04.005

[5] QUDT.org. "Qudt: Quantities, units, dimensions and types," Accessed: Aug. 31, 2026. [Online]. Available: https://www.qudt.org/

[6] H. Knublauch and D. Kontokostas, *Shapes constraint language (shacl)*, W3C Recommendation, 2017. [Online]. Available: https://www.w3.org/TR/shacl/

[7] Prefix.dev, *Pixi installation*. Accessed: Sep. 26, 2026. [Online]. Available: https://pixi.prefix.dev/latest/installation/

[8] H. Knublauch and D. Kontokostas, "Shapes constraint language (shacl)," World Wide Web Consortium, W3C Recommendation, Jul. 2017. Accessed: Aug. 31, 2026. [Online]. Available: https://www.w3.org/TR/shacl/